🛠️ Smart Attendance System

User Guide & Deployment Documentation

Production — uniquejob.ca — v1.0

📜 Table of Contents

1. System Overview
2. Server Access & Remote Desktop
3. Application Routes & URLs
4. Docker Services & Architecture
5. Admin Panel Usage
6. phpMyAdmin — Database Management
7. Redis Commander — Cache Management
8. Deployment Guide — Code Changes
9. Database Migrations
10. Viewing Logs & Troubleshooting
11. Enabling HTTPS / SSL
12. Backup & Restore
13. Quick Reference & Cheat Sheet

1. System Overview

The **Smart Attendance Management System** is a production-ready web application for managing biometric/attendance devices, employees, departments, shifts, and access control. The system is designed to communicate with physical attendance devices via WebSocket connections.

Technology Stack

Component	Technology
Backend Framework	Django 6.0.1 (Python)
ASGI Server	Daphne (async WebSocket + HTTP)
Database	MySQL 8.4
Cache / Message Broker	Redis 7.4
Background Tasks	Celery Worker + Celery Beat
Device Communication	Custom Device Broker (WebSocket)
Web Server / Proxy	Nginx 1.28
Containerization	Docker + Docker Compose
SSL	Let's Encrypt (Certbot)
OS	Ubuntu (KVM VPS)

Server Details

Property	Value
Public IP	`187.77.10.40`
Domain	`uniquejob.ca`
SSH Port	`22`
RDP Port	`3389`
Project Path	`/home/Attendance-management/WebSocketSDK_Python/`

2. Server Access & Remote Desktop

💻 SSH Access (Command Line)

```
# Open a terminal and connect via SSH
ssh root@187.77.10.40
```

🖥️ Remote Desktop (GUI Access)

A full graphical desktop environment is available via **Remote Desktop Protocol (RDP)**. You can use this to access the server with a GUI — browse files, open a browser, manage services visually, etc.

1. **Open the Run Dialog**
   On your Windows PC, press `Win + R` to open the Run dialog.
2. **Type `mstsc` and press Enter**
   This opens the *Remote Desktop Connection* application.
3. **Enter Connection Details**

Computer	`187.77.10.40`
Username	`devuser`
Password	Use the **same server password** provided to you

4. **Click Connect**
   Accept the certificate warning if prompted. You are now connected to the server's graphical desktop.

> 💡 **Tip:** If the RDP connection is slow, reduce the display resolution and color depth in the Remote Desktop settings before connecting (click *Show Options → Display*).

3. Application Routes & URLs

All services are accessible via the domain **http://uniquejob.ca** (or via the server IP **http://187.77.10.40**). Nginx handles routing to the appropriate backend service.

URL	Service	Description
`http://uniquejob.ca/`	Django App	Main login page → redirects to `/ui/login`
`/ui/`	Django App	Main web UI — devices, users, attendance, reports
`/admin/`	Django Admin	Django admin panel (superuser access)
`/api/`	REST API	Django REST Framework API endpoints
`/phpmyadmin/`	phpMyAdmin	MySQL database management UI
`/redis/`	Redis Commander	Redis cache browser & management UI
`/ws/`	Device Broker	WebSocket endpoint for attendance devices
`/static/`	Nginx (direct)	Static files — CSS, JS, images
`/media/`	Nginx (direct)	User-uploaded media — photos, profiles

> ⚠️ **Note — HTTPS:** Currently the site runs on **HTTP (port 80)**. Port 443 (HTTPS) is blocked at the hosting provider level. Once the provider opens port 443, HTTPS will automatically be available via the already-configured SSL certificates. See Section 11.

Direct Docker Ports (Internal / Advanced)

These ports are accessible directly on the server IP for debugging. They bypass Nginx.

URL	Service	Port
`http://187.77.10.40:8082`	Django App (direct)	8082
`http://187.77.10.40:8080`	phpMyAdmin (direct)	8080
`http://187.77.10.40:8081`	Redis Commander (direct)	8081
`ws://187.77.10.40:8001`	Device Broker (direct)	8001
`tcp://187.77.10.40:3307`	MySQL	3307
`tcp://187.77.10.40:6380`	Redis	6380

4. Docker Services & Architecture

The entire system runs inside **8 Docker containers** managed by Docker Compose.

🌐 **webapp**
Django + Daphne ASGI server. The main attendance application.
Port 8082

📡 **devicebroker**
WebSocket broker for device communication.
Port 8001 / 8002

🗂️ **mysql**
MySQL 8.4 — primary database for all application data.
Port 3307 → 3306

⚡ **redis**
Redis 7.4 — caching, sessions, Celery message broker.
Port 6380 → 6379

🔧 **celery-worker**
Processes async background tasks (email, syncing, etc.).
Internal only

⏰ **celery-beat**
Scheduler — triggers periodic/cron tasks.
Internal only

📊 **phpmyadmin**
Web-based MySQL management — browse tables, run SQL.
Port 8080

🔎 **redis-commander**
Web-based Redis browser — inspect keys, values, TTL.
Port 8081

Viewing Container Status

```
# Navigate to the project directory
cd /home/Attendance-management/WebSocketSDK_Python/

# View all running containers
docker compose ps

# Check container health & resource usage
docker stats --no-stream
```

5. Admin Panel Usage

The Django Admin panel provides full control over all system data.

Property	Value
URL	`http://uniquejob.ca/admin/`
Username	`admin`
Password	`admin123`

> 🚨 **Security:** Change the default admin password immediately in production! Go to `/admin/` → Users → admin → Change password.

Key Admin Sections

- **Devices** — Add/edit/remove attendance devices, view connection status
- **Device Users** — Manage employees enrolled on devices
- **Departments** — Organizational structure
- **Attendance Logs** — View punch-in/punch-out records
- **Access Control** — Door access rules and time zones
- **Firmware** — Manage device firmware updates

6. phpMyAdmin — Database Management

phpMyAdmin provides a visual interface to manage the MySQL database — browse tables, run SQL queries, import/export data.

Property	Value
URL	`http://uniquejob.ca/phpmyadmin/`
Server	`mysql` (pre-configured)
Username	`attendance_user`
Password	As set in `docker-compose.yml` (MYSQL_PASSWORD)
Database	`attendance_db`

> 💡 **Tip:** You can also connect to MySQL from the command line:
> `docker compose exec mysql mysql -u attendance_user -p attendance_db`

Common Tasks

- **Browse Data** — Click a table name in the left sidebar to view its records
- **Run SQL** — Click the "SQL" tab to execute custom queries
- **Export** — Click "Export" to download a database backup (.sql file)
- **Import** — Click "Import" to restore from a .sql backup file

7. Redis Commander — Cache Management

Redis Commander lets you visually inspect and manage the Redis cache — view keys, values, TTL, and flush data when needed.

Property	Value
URL	`http://uniquejob.ca/redis/`
Connection	Auto-configured to cluster Redis

What You'll See

- **Keys** — Session data, Celery task results, cached queries
- **TTL** — Time-to-live for each key
- **Memory** — Redis memory usage stats

8. Deployment Guide — Making Code Changes

When you need to update the application code (fix a bug, add a feature, change a template), follow these steps to re-deploy with **minimal downtime**.

⚡ Quick Deploy (Most Common)

For changes to Python code, templates, or static files:

```
# 1. Navigate to the project directory
cd /home/Attendance-management/WebSocketSDK_Python/

# 2. Make your code changes (edit files via RDP, nano, vim, or VS Code)

# 3. Collect static files (if CSS/JS/images changed)
docker compose exec webapp python DjangoExample/manage.py collectstatic --noinput

# 4. Rebuild and restart ONLY the webapp container
docker compose up --build -d webapp

# 5. Verify it's running
docker compose ps
# ✓ attendance_webapp should show "Up" status
```

> ✅ **Zero-downtime:** Only the webapp container restarts. MySQL, Redis, and the device broker keep running. Typical restart takes 5–10 seconds.

🛠️ Full Rebuild (Major Changes)

For changes to `requirements.txt`, `Dockerfile`, or `docker-compose.yml`:

```
# Rebuild ALL containers from scratch
cd /home/Attendance-management/WebSocketSDK_Python/

docker compose down
docker compose up --build -d

# Wait for all containers to become healthy
docker compose ps

# Check logs if something isn't starting
docker compose logs webapp --tail 50
```

📂 Updating Specific Services

```
# Restart only the device broker
docker compose up --build -d devicebroker

# Restart only the celery workers
docker compose up --build -d celery-worker celery-beat

# Restart Redis or MySQL (data persists via volumes)
docker compose restart redis
docker compose restart mysql
```

📄 Updating Static Files Only

```
# After editing CSS, JS, or image files:
docker compose exec webapp python DjangoExample/manage.py collectstatic --noinput

# Clear browser cache or hard-refresh (Ctrl+Shift+R)
```

9. Database Migrations

When you add/change a model (fields, tables) in the Django code, you must create and apply **migrations** to update the database schema.

Step-by-Step

```
# 1. Make code changes to models.py

# 2. Generate migration files
docker compose exec webapp python DjangoExample/manage.py makemigrations

# 3. Review the generated migration (optional but recommended)
docker compose exec webapp python DjangoExample/manage.py showmigrations

# 4. Apply migrations to the database
docker compose exec webapp python DjangoExample/manage.py migrate

# ✓ Database schema is now updated
```

> ⚠️ **Important:** Always back up the database before running migrations in production!
> `docker compose exec mysql mysqldump -u attendance_user -p attendance_db > backup_$(date +%Y%m%d).sql`

Common Migration Commands

Command	Description
`makemigrations`	Generate new migration files from model changes
`migrate`	Apply pending migrations to the database
`showmigrations`	List all migrations and their applied status
`migrate app_name zero`	Rollback all migrations for a specific app
`sqlmigrate app_name 0001`	Show the SQL that a migration would execute

Creating a New Superuser

```
docker compose exec webapp python DjangoExample/manage.py createsuperuser
```

10. Viewing Logs & Troubleshooting

Docker Container Logs

```
# View logs for a specific service
docker compose logs webapp --tail 100 -f
docker compose logs devicebroker --tail 50
docker compose logs celery-worker --tail 50
docker compose logs mysql --tail 50

# View ALL container logs at once
docker compose logs --tail 50 -f

# -f = follow (live output)   --tail 100 = last 100 lines
```

Nginx Logs

```
# Access log — every HTTP request
tail -f /var/log/nginx/attendance_access.log

# Error log — proxy errors, upstream timeouts
tail -f /var/log/nginx/attendance_error.log
```

Common Issues & Fixes

Problem	Solution
502 Bad Gateway	Webapp container is down. Run `docker compose up -d webapp`
Static files not loading	Run `docker compose exec webapp python DjangoExample/manage.py collectstatic --noinput`
Database connection error	Check MySQL: `docker compose ps mysql` — wait for "healthy" status
Device page hangs	Restart the device broker: `docker compose restart devicebroker`
Celery tasks not running	Restart workers: `docker compose restart celery-worker celery-beat`
Disk space full	Clean Docker: `docker system prune -a --volumes` (⚠ removes unused data)

11. Enabling HTTPS / SSL

SSL certificates from **Let's Encrypt** are already installed and configured. The HTTPS server block in Nginx is ready. The only blocker is that **port 443 is currently closed at the hosting provider level**.

To Enable HTTPS

1. **Open Port 443 at the Hosting Provider**
   Log into your hosting control panel (e.g., Contabo at `https://my.contabo.com`) → VPS settings → Firewall → Add rule to allow **TCP port 443** inbound.
2. **Update the Nginx Config**
   Edit `/etc/nginx/sites-enabled/uniquejob.ca` and change the HTTP server block to redirect to HTTPS:

```
# In the HTTP (port 80) server block, replace the location / block with:
location / {
    return 301 https://uniquejob.ca$request_uri;
}
```

3. **Test & Reload Nginx**

```
nginx -t && nginx -s reload
```

Renewing SSL Certificates

```
# Let's Encrypt certs expire every 90 days. Renew with:
certbot renew --dry-run   # Test renewal
certbot renew             # Actually renew

# Auto-renewal is typically set up via cron/systemd timer
systemctl list-timers | grep certbot
```

12. Backup & Restore

Database Backup

```
# Export full database to SQL file
docker compose exec mysql mysqldump -u attendance_user -p attendance_db > /home/Attendance-management/b

# Create backups directory if it doesn't exist
mkdir -p /home/Attendance-management/backups/
```

Database Restore

```
# Restore from a backup file
docker compose exec -T mysql mysql -u attendance_user -p attendance_db < /home/Attendance-management/ba
```

Media Files Backup

```
# Backup uploaded media (photos, profiles, etc.)
tar -czf /home/Attendance-management/backups/media_$(date +%Y%m%d).tar.gz \
  /home/Attendance-management/WebSocketSDK_Python/DjangoExample/demosite/media/
```

Full Project Backup

```
# Backup the entire project directory
tar -czf /home/Attendance-management/backups/full_project_$(date +%Y%m%d).tar.gz \
  /home/Attendance-management/WebSocketSDK_Python/
```

13. Quick Reference — Cheat Sheet

🏠 Essential URLs

What	URL
Main Application	`http://uniquejob.ca/`
Admin Panel	`http://uniquejob.ca/admin/`
REST API	`http://uniquejob.ca/api/`
phpMyAdmin	`http://uniquejob.ca/phpmyadmin/`
Redis Commander	`http://uniquejob.ca/redis/`

💻 Essential Commands

```
# ── NAVIGATE ──────────────────────────────────────
cd /home/Attendance-management/WebSocketSDK_Python/

# ── DOCKER BASICS ─────────────────────────────────
docker compose ps                        # List all containers
docker compose up -d                     # Start all services
docker compose down                      # Stop all services
docker compose restart webapp            # Restart one service
docker compose logs webapp --tail 50 -f  # View live logs

# ── DEPLOY CODE CHANGES ───────────────────────────
docker compose up --build -d webapp      # Rebuild & restart webapp

# ── DJANGO MANAGEMENT ─────────────────────────────
docker compose exec webapp python DjangoExample/manage.py migrate
docker compose exec webapp python DjangoExample/manage.py collectstatic --noinput
docker compose exec webapp python DjangoExample/manage.py createsuperuser
docker compose exec webapp python DjangoExample/manage.py makemigrations
docker compose exec webapp python DjangoExample/manage.py shell

# ── NGINX ─────────────────────────────────────────
nginx -t                                 # Test config syntax
nginx -s reload                          # Reload config (no downtime)
tail -f /var/log/nginx/attendance_error.log

# ── FIREWALL ──────────────────────────────────────
ufw status                               # View firewall rules
ufw allow 80/tcp                         # Open a port
ufw deny 3307/tcp                        # Block a port

# ── DATABASE BACKUP ───────────────────────────────
docker compose exec mysql mysqldump -u attendance_user -p attendance_db > backup.sql
```

🛠️ File Locations

File / Directory	Purpose
`/home/Attendance-management/WebSocketSDK_Python/`	Project root
`docker-compose.yml`	Docker services configuration
`Dockerfile`	Build instructions for Python containers
`DjangoExample/`	Django application directory
`DjangoExample/demosite/settings/`	Django settings (base, dev, production)
`DjangoExample/sdkdemoapp/models.py`	Database models
`DjangoExample/sdkdemoapp/views.py`	View logic / controllers
`DjangoExample/sdkdemoapp/templates/`	HTML templates
`DjangoExample/sdkdemoapp/static/`	App-level static assets
`DjangoExample/demosite/staticfiles/`	Collected static files (served by Nginx)
`DjangoExample/demosite/media/`	User uploads (photos, etc.)
`packages/devicebroker/`	Device broker source code
`/etc/nginx/sites-enabled/uniquejob.ca`	Nginx reverse proxy config
`/etc/letsencrypt/live/uniquejob.ca/`	SSL certificates

🔐 Credentials Summary

Service	Username	Password
Django Admin	`admin`	`admin123` (change this!)
RDP (Remote Desktop)	`devuser`	Server password
SSH	`root`	Server password
MySQL (phpMyAdmin)	`attendance_user`	See docker-compose.yml

Smart Attendance Management System — Production Deployment Guide
Server: 187.77.10.40 | Domain: uniquejob.ca | Generated: February 2026
📄 This document can be printed or saved as PDF via **Ctrl+P** → Save as PDF